

AT1 Part B – KMNIST Digit Classification Task

Dr. Christopher Özbek - 26239780

Abstract

A PyTorch neural network was trained to recognize cursive Japanese Kuzushiji characters from the KMNIST Dataset (Clanuwat, 2018). Various experiments were performed to improve accuracy and the final configuration of 3 hidden layers (512 neurons each), dropout of 0.2 on the first layer and simple image-based data augmentation achieved an accuracy of 92.2% and not too much overfitting after 208 epochs.

1 Introduction

The modern Japanese kanji-based writing system was only formalized during the Meiji restoration in 1868. In documents before this time, the handwritten Kuzushiji kanjis could be written in various variant forms (cf. Figure 1). The KMNIST dataset (Clanuwat, 2018) was originally compiled and published to exhibit this diverseness and offers a machine learning task of mapping gray-scale images of 28 by 28 pixels to the correct Kanji character.

1.1 Data presentation

The KMNIST dataset consists of 70,000 images representing 10 kanji characters. In contrast to the original MNIST dataset (Deng, 2012) these 10 characters do not represent the numbers 0 to 9 but the 10 syllabic characters shown in Table 1. Each character is represented as white pixels on black background on a 28 x 28 pixels image using an 8-bit grayscale palette. The dataset was provided for this assignment already split into 60,000 images and associated labels for training and 10,000 images and labels for testing.

0: お (o)	1: き (ki)	2: す (su)	3: つ (tsu)	4: な (na)
5: は (ha)	6: ま (ma)	7: や (ya)	8: れ (re)	9: を (wo)

Table 1: Target Classes in modern Kanji representation

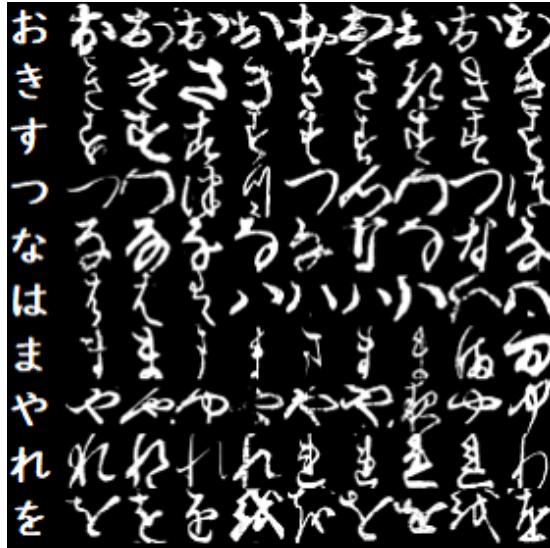


Figure 1: Example images for each of the classes from the dataset (from Clanuwat, 2018).
Modern kanji shown on the left, historic Kuzushiji examples in the row to the right.

1.2 Neural Network Architecture

In line with the assignment and progress of the course lecture (as of 28.02.2026), I used exclusively fully-connected linear hidden layers (`torch.nn.Linear`). I started with a most basic design of using a single layer hidden dimension with 512 neurons as my baseline setup.

In an initial, exploratory phase I ran various experiments to determine which hyperparameter would help me most attain the business objective of 80% accuracy without much overfitting. I adjusted learning rate (increasing and decreasing from the default of $lr = 0.001$), adding additional layers of 512, 128 and 64 in various depths and adding data augmentation based on `torchvision.transforms`. These experiments were only loosely documented and can be found in the technical appendix at the bottom of this report. In retrospect, I would say I wasn't able to form an intuition on the effect of most parameters. In particular, their interaction with each other has eluded me. For instance, my initial tests showed that a learning rate of 0.0005 (instead of 0.001 would improve results), but this didn't lead to better outcomes in my final set of experiments. Substantially more computation time would be necessary to assess the various combinations which wasn't feasible under the time constraints for this assignment.

At the end of the exploratory phase, I kept the single hidden layer with 512 neurons, AdamW as the optimizer with default settings of $lr=0.001$ as a starting point for more formal experimentation. I didn't observe a downside of increasing batch size to 1024 (from 128), but considerable speed-up during GPU training. I thus adopted this for my experiments.

Training was performed for up to 500 epochs with early stopping if training loss didn't improve by at least 0.1% over 20 epochs or when loss was below 0.001. Early stopping was employed to speed up experiments and conserve computation resources.

Following the exploratory phase, I performed 3 formal experiments detailed in my submitted notebook and explained below to improve over the baseline. Table 2 shows a summary of these formal experiments.

Experiment	Hidden Layers	Epochs / Batch	Train Loss / Test Loss	Accuracy on test
#0 – Baseline	[512]	77 / 1024	0.0010 / 0.4911	90.9%
#1 - Data Augmentation	[512]	170 / 1024	0.1053 / 0.2292	94.2%
#2 – Dropout	[512] with 20% Dropout	150 / 1024	0.1607 / 0.2324	93.6%
#3 – Deeper Layout	[512, 512, 512] with 20% Dropout in the first layer	200 / 1024	0.0568 / 0.1407	96.4%

Table 2: Experiment overview. All experiments were performed with default learning rate of 0.001 with AdamW optimizer until loss didn't improve for 0.1% over 20 epochs.

1.2.1 Baseline experiment #0

When using a single 512 neuron hidden layer, the neural network reaches overfitting within less than 10 epochs. Test loss continuously rises, while accuracy plateaus at 91.0%. After 77 epochs it terminated because training loss decreased below 0.001 which was set as a stopping criterion. Performance evaluation showed that the model had attained 100% accuracy on the training set further corroborating the overfitting result.

The confusion matrix (Fig 3) showed false predictions scattered over all target/prediction pairs with the top 3 failure pairs being: Label 2 (す Su) predicted 48 times as 3 (つTsu), Label 5 (は ha) predicted 41 times as 2 (す su) and Label 1 (き Ki) predicted 40 times as 6 (ま Ma). The mispredictions between Ki and Ma looks plausible because the Kanji shape with two horizontal bars, one horizontal stem and a swirl at the bottom invite misclassification. On the other hand the misprediction between Su vs Tsu struck me as odd, because the modern kanji shapes are very distinct. Exploring various misprediction pairs showed that the Tsu Kuzushiji kanji is very variant rich and does not follow the same stroke pattern as in the modern つTsu kanji. An example is shown in Figure 4.



Figure 2: Baseline experiment #0 with single layer of 512 neurons. Shows signs of overfitting.

Confusion Matrix for Baseline (Simple) Model on Test Dataset

0	928	4	3	2	16	12	3	14	16	2
1	1	885	18	2	12	10	40	6	16	10
2	7	5	876	48	10	17	6	9	14	8
3	3	3	26	947	0	7	5	3	4	2
4	19	22	9	10	881	7	16	6	20	10
5	4	10	41	8	5	903	17	1	7	4
6	3	7	27	10	13	3	933	2	0	2
7	8	11	17	2	11	4	14	905	13	15
8	3	13	16	19	2	5	6	1	930	5
9	9	7	19	9	18	3	9	5	17	904
	0	1	2	3	4	5	6	7	8	9

Predicted labels

Figure 3: Confusion matrix for baseline experiment #0

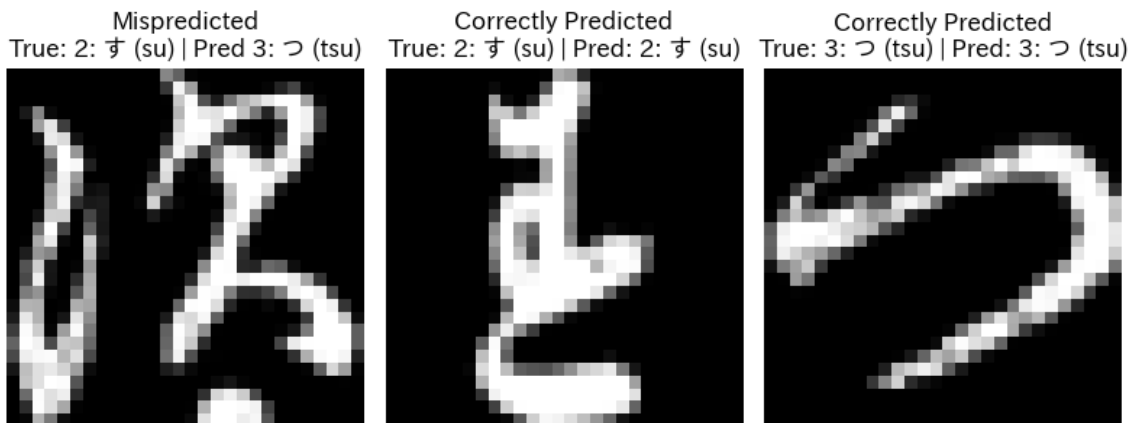


Figure 4 : Example of misprediction by the #0 Base-model

1.2.2 Data augmentation #1

To prevent overfitting one might consider data augmentation, regularization and dropout as orthogonal strategies to explore. I decided to first apply data augmentation, because the image domain of the test data and the small overall size of the test set made it seem a good approach to pursue. I used torchvision transforms to rotate the images by +/- 10 degrees, translate them by +/- 3 pixel, scale them to 90% or 110% of their size and adjust brightness by +/- 30% and contrast by +/- 10%. An example augmentation is shown in Figure 5. Data augmentation was applied in 90% of training examples to retain unmodified original images in the dataset at least 10% of the time.

Results shows that accuracy improves to 94.2% and overfitting is clearly reduced (see Figure 6).

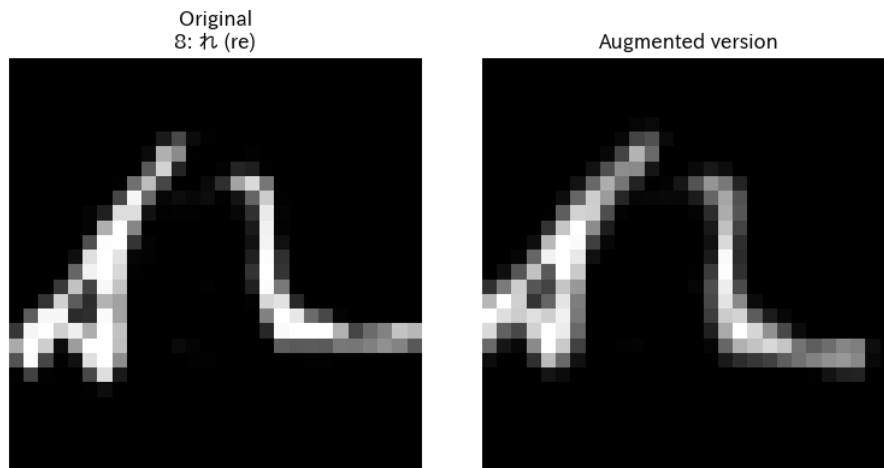


Figure 5: Example for the data augmentation performed. Note the slight rotation in clockwise orientation and the shift by two pixels to the left.



Figure 6: Training performance of model #1 with 512 neurons hidden layer and data augmentation on 90% of training runs.

1.2.3 Dropout experiment #2

I further modified the network to include a dropout of 20% after the 512-neuron hidden layer and retain the data augmentation explored in model #1. In comparison with the previous experiment #1 with just data augmentation, the training progresses more slowly and the training terminates 20 epochs earlier than with experiment #1. Attained accuracy is 93.6% which is sub-par to using just data augmentation as in experiment #1.

In comparison to the final training loss of 0.1053 and testing loss of 0.2292 for the model #1 without dropout, the gap between training and testing loss decreased significantly (0.1607 training loss vs 0.2324 testing loss) in line with our expectations for the benefits of using dropout.

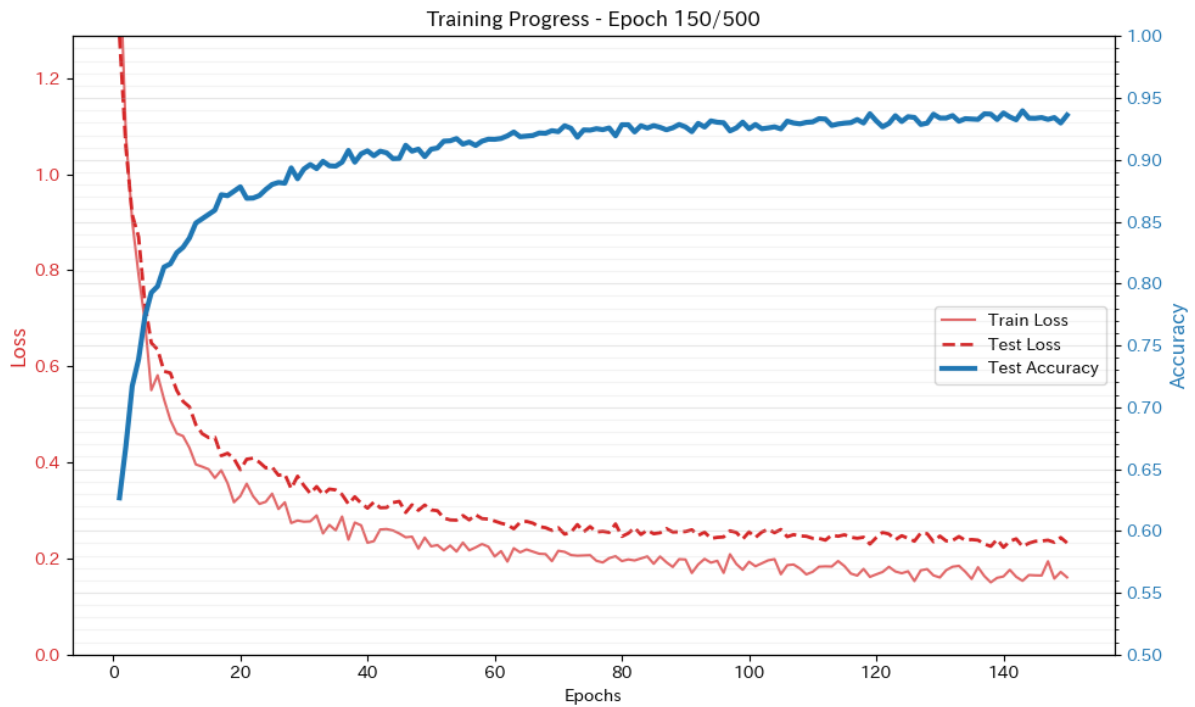


Figure 7: Performance of experiment #2 with added dropout of 20% after the hidden layer.

1.2.4 Experiment 3: Using a deeper network

So far, the network employed was very shallow. Despite having roughly 400,000 parameters ($768 * 512 + 512 * 10$) this might not be enough to really learn the complex shapes of Kanjis and generalize them. As a last experiment, I thus add 2 additional hidden layers of 512 neurons each.

This last experiment reached a final accuracy of 96.4% after 200 epochs with training loss of 0.0568 and testing loss of 0.1407 as shown in Figure 8. Evaluation of the confusion matrix (shown in Figure 9) shows that Su vs Tsu misclassifications remain high at 38 mispredictions (vs 41 in the baseline model), but all other mispredictions are significantly reduced and the second most likely misprediction of the baseline model (Label 5 (は) predicted 2 (す su)) is only mispredicted 8 times in comparison to previous 40 times.



Figure 8: Training chart of model #3 showing superior performance over all other models. Overfitting is significantly reduced.

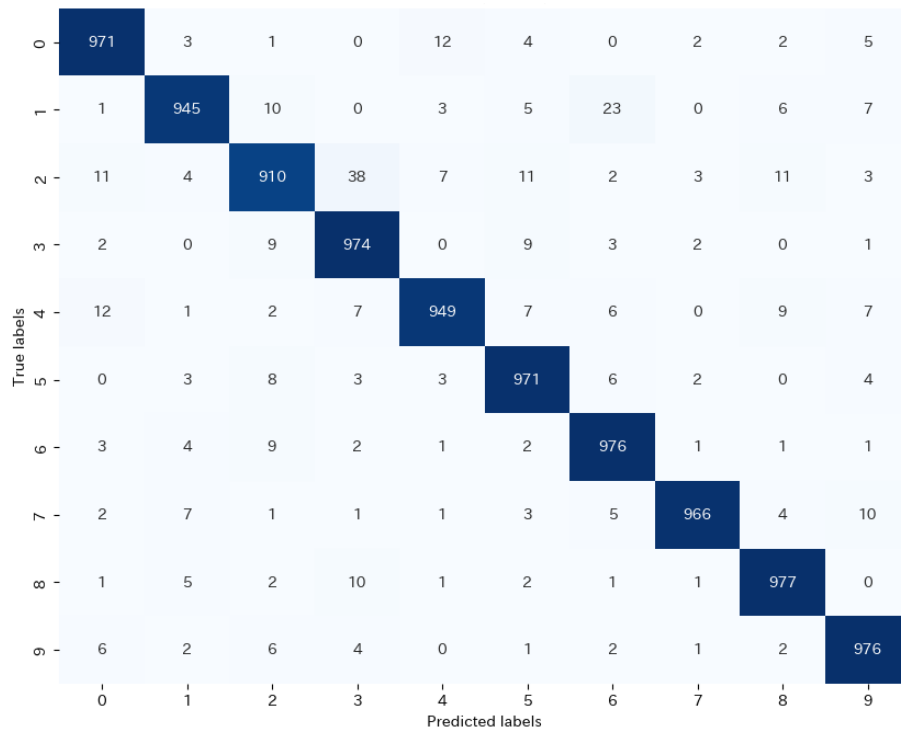


Figure 9: Confusion matrix for experiment #3 Deeper Network. Mispredictions are reduced roughly by half in comparison to the baseline model (9.1% misprediction in baseline model vs 4.6% in experiment #3).

1.3 Discussion model performance and limitations

The task of classifying 10 Kuzushiji characters from 28x28 greyscale images was explored in the preceding experiments. Baseline performance of employing a single 512-neuron hidden layer was already fulfilling the business objective of 80% accuracy albeit suffered from severe overfitting. To strengthen the model's ability to generalize and reduce overfitting both data augmentation and dropouts were successfully used as strategies.

Overall accuracy improved from 90.9% in the baseline model to 96.4% in the final experiment (3 hidden layers of 512 neurons each, with 20% dropout in the first layer and data augmentation during training). Final training loss was 0.0568 and final test loss was 0.1407. In comparison to the model performance described in the KMNIST publication (Clanuwat, 2018), our model exceeds the performance of the 2-layer convolutional network design using Keras (95.1%) but falls short of the performance of an 18-layer ResNet (97.8%).

While the business objectives are fulfilled, our experiments have several limitations:

- Our preceding experiments were limited by unavailability of GPU resources after Colab compute credits were exhausted. We had to rely on early and opportunistic stopping rules to abort training early. Additional training, for instance up to 1,000 or 5,000 epochs, might further improve results by 1-2% based on the existing network design.
- Limited time resources prevented us from further exploring alternative network designs. Adding two hidden layers improved performance by 3% in the last experiment. Potentially additional layers might increase performance even further.
- We haven't explored any other hyperparameters available to the PyTorch training process. In particular, optimizer parameters such as optimization algorithm, learning rate or weight decay were left unexplored in formal testing.
- Training and testing were only performed on the provided data set. Biases, errors and omissions in the training and test data set might have caused misleading results in real world performance detecting historic kanji characters. Mislabeling by the original data authors will cause the model to learn wrong classifications. Errors in the test data set might lead to wrong performance numbers, giving us false confidence in the capabilities of the model.

1.4 Remaining issues and recommendations

The resulting model still misclassifies roughly 1 in 25 characters and lags behind the models discussed in the KMNIST publication itself, which achieve up to 98.83% accuracy using a PreActResNet-18 + Manifold Mixup (Verma 2018).

Before putting this model into production, the implications of the remaining misclassifications should be explored qualitatively and additional real-world images should be obtained to assess performance on third-party acquired text. Insufficient performance by a machine learning model used on historical text could for instance lead to mistranscription which might in turn cause misinterpretation of historical facts. The ethical implications of model usage should thus be considered.

Model performance is only based on characters preprocessed in a pipeline mostly identical to the original KMNIST publication. Characters must thus be precisely cropped, aligned, centered, scaled to fit into 28x28 greyscale images.

Further optimization was prevented by lack of access to more compute resources as Google Colab's free tier does not permit sufficient resources for extensive experimentation and prolonged training. In particular, I wasn't able to explore the role of additional experimentation

with the hyperparameters learning rate, dropout, L1/L2 regularization and form an intuition about the benefits of changing the model architecture.

For future work, the following recommendations are made:

- Continue and expand experimentation using capable hardware to explore the hyperparameter space more extensively. A structured approach of documenting and insulating the effects on training performance would be required.
- Expand the included data for instance by training to include KMNIST-49 (all 49 Kuzushiji characters) to force the model to be more general in its classification abilities.
- Add further experiments which use existing base models and architectures for image classification and only fine-tune them on the given task.

Bibliography

Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., & Ha, D. (2018). Deep Learning for Classical Japanese Literature. ArXiv, abs/1812.01718.

Deng, L. (2012). The MNIST database of handwritten digit images for machine learning research. IEEE Signal Processing Magazine, 29(6), 141–142.

Verma, V., Lamb, A., Beckham, C., Najafi, A., Courville, A., Mitliagkas, I., & Bengio, Y. (2018). Manifold Mixup: Learning Better Representations by Interpolating Hidden States. ArXiv, abs/1806.05236.

W. So, A. So, et al. (2026), UTS Deep Learning, Autumn 2026, Slides and Exercises

Technical Appendix:

I ran the following exploratory experiments which were superseded by the formal experiments noted in the Jupyter notebook:

Experiment Focus	Configuration Notes	Epochs / Batch	Final Test Acc.	Observation / Result
Augmentation & Shallow	Affine (Rot 15 deg, Scale 0.9-1.1, Trans 0.1). 1x64 layer, 10% dropout at end.	300 / 4096	94.2%	Baseline for exploration. (Loss: Train 0.109, Test 0.215)
Depth (+1 Layer)	Added second 64-neuron hidden layer. Affine (Rot 10 degree, Trans 0.05).	300 / 4096	95.1%	Performance improved with extra depth.
Depth (+2 Layers)	Added third 64-neuron hidden layer.	300 / 4096	94.6%	Performance dropped slightly. Indicated potential overfitting.
Color Jitter	Added Color Jitter. (Continued from previous weights).	N/A	95.4%	Introducing color noise improved generalization.
Heavy Jitter	Increased Color Jitter to 30% brightness/contrast.	N/A	94.9%	Too much color noise slightly degraded accuracy.
Depth & Dropout shift	Added fourth 64-neuron layer. Moved Dropout to the front.	300 / 4096	95.6%	Moving dropout to the first layer yielded the best result so far.
High Learning Rate	Increased lr from 0.001 to 0.01.	300 / 4096	94.8%	Converged poorly compared to default learning rate.
Low Learning Rate	Decreased lr to 0.0005.	300 / 4096	95.2%	Learning was still ongoing at epoch 300; requires more epochs.
Large Batch + Low LR	lr 0.0005, Batch 8192, Affine scaling 0.6-1.1.	300 / 8192	93.9%	Huge batch size combined with low LR hindered learning.
Massive Epoch Run	Deep architecture run for 5000 epochs.	5000 / 4096	96.5%	Maximum baseline reached (Train loss: 0.046, Test loss: 0.155).